LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL4 Enkapsulasi



Azizatul Ainy

103132400030

S1DS REG 05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Java Modifier

Modifier memberi dampak tertentu pada kelas, *interface*, dan variabel. Java Modifier dibagai menjadi kelompok;

a. Access modifier berlaku untuk kelas, *method*, dan variabel, meliputi *modifier public, protected, private,* dan tak ada modifier.
b. Final modifier berlaku untuk kelas, *method*, dan variabel, meliputi *modifier* final.
c. Static modifier berlaku untuk *method*, dan variabel, meliputi *modifier* static.
d. Abstract modifier berlaku untuk kelas, dan *method*, meliputi *modifier* abstract.
e. Syncrinized modifier berlaku untuk *method*, meliputi *modifier* syncronized.
f. Native modifier berlaku untuk *method*, meliputi *modifier* native.
g. Storage modifier berlaku untuk variabel, meliputi *modifier* transient dan volatile.

Deklarasi modifier dalam Java:

```
/*deklarasi modifier di class/interface*/
   [Modifier] class/interface [nama class/interface] {}

//deklarasi modifier di atribut
   [Modifier][TypeData][nama Atribut];

//deklarasi modifier di Method
   [Modifier]{[TypeData]}[nama Method]
   (parameter_1,parameter_2,parameter_n){}
```

Kesepakatan umum penamaan dalam suatu kelas:

- Nama Kelas : gunakan kata benda dan huruf pertama dari tiap kata ditulis dengan huruf besar dan memiliki access modifier public.
- Pada umumnya, atribut diberi access modifier private, dan method diberi access modifier public.
- Nama atribut : gunakan kata benda, dan diawali dengan huruf kecil : nama, umur
- Nama Method : gunakan kata kerja; kecuali huruf yang pertama, huruf awal tiap kata ditulis kapital : getNama(), getUmur()
- Untuk method yang akan memberikan atau mengubah nilai dari suatu atribut, nama method Kita tambahi dengan kata kunci ”set”: setUmur(int a),setNama(String a)
- Untuk Method yang akan mengambil nilai dari atribut, nama Method Kita tambahi dengan kata kunci ”get” : getUmur(), getNama()
- Konstanta. Semuanya ditulis dengan huruf besar; pemisah antar kata menggunakan garis bawah: MAX_VALUE, DECIMAL_DIGIT_NUMBER

2. Constructor

Constructor adalah tipe khusus method yang digunakan untuk menginstansiasi atau menciptakan sebuah objek. Nama *constructor* adalah sama dengan nama kelasnya. Selain itu, *constructor* tidak bisa mengembalikan suatu nilai bahkan *void* sekalipun. *Constructor* juga dimanfaatkan untuk membangun suatu objek dengan langsung mengeset atribut-atribut yang disandingkan pada objek yang dibuat tersebut.

Access *modifier* yang digunakan pada constructor selayaknya adalah *public* karena *constructor* tersebut akan diakses diluar kelasnya. Cara memanggil *constructor* adalah dengan menambahkan *keyword new*.
Deklarasi *constructor*;

```
[modifier] namaclass(parameter1){
        Body constructor;
}
[modifier] namaclass(parameter1,parameter2){
        Body constructor;
}
[modifier] namaclass(parameter1,parameter2,...,parameterN){
        Body constructor;
}
```

3. Package

Package digunakan untuk mengatur *class* yang telah dibuat dan akan bermanfaat jiks *class* yang dibuat sangat banyak sehingga perlu dekelompokkan berdasarkan kategori tertentu. Selain pengelompokkan *class*, penggunaan package dapat mengatasi konflik penamaan *method* jika suatu *method* ditulis dalam *package* yang berbeda.
Terdapat dua cara menggunakan suatu package;
a. *Class* yang menggunakan berada dalam direktori (*package*) yang sama dengan *Class* yang digunakan, Maka tidak diperlukan *import*.
b. *Class* yang menggunakan berada dalam direktori (*package*) yang berbeda dengan *Class* yang digunakan, Maka perlu dilakukan *import package*.

B. Guided

1. Modifier & Constructor

```
package Guided01;

public class Manusia {
//definisi atribut
private String nama;
private int umur;

//4.2 sebuah constructor
public Manusia() {}

public Manusia(String nama) {
    this.nama = nama;
}

public Manusia(String nama, int umur) {
    this.nama = nama;
    this.umur = umur;
}
//mahasiswaobjekManusia = new Manusia(
//Method setter
public void setNama(String a){
    nama = a;
}
//method getter
public String getNama(){
    return nama;
}
public void setUmur(int umur){
    this.umur = umur;
}
public int getUmur(){
    return umur;
}
}
```

```
package Guided01;

public class Demomanusia {
    public static void main(String[] args) {
        Manusia arrMns[] = new Manusia[3];

        // Constructor pertama
        Manusia objMns1 = new Manusia();

        // kedua
```

```
        Manusia objMns2 = new Manusia("John");

        // Ketiga
        Manusia objMns3 = new Manusia("Bajuri", 44);

        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;

        for (int i = 0; i < 3; i++) {
            System.out.println("Nama : " + arrMns[i].getNama());
            System.out.println("Umur : " + arrMns[i].getUmur());
            System.out.println();
        }
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Enkapsulasi ---
Nama : null
Umur : 0

Nama : John
Umur : 0

Nama : Bajuri
Umur : 44

------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.677 s
Finished at: 2026-10-12T22:01:51+07:00
```

2. Package

```
package Guided2;

public class Hargapulsa {
   public void info(){
System.out.println("Harga Pulsa");
System.out.println("Rp. 5000");
System.out.println("Rp. 10000");
System.out.println("Rp. 20000");
System.out.println("Rp. 50000");
System.out.println("Rp. 100000");
}
}
```

```
package Guided2;
```

```
public class Hargatoken {
  public void info(){
System.out.println("Harga Token");
System.out.println("Rp. 20000");
System.out.println("Rp. 50000");
System.out.println("Rp. 100000");
System.out.println("Rp. 200000");
System.out.println("Rp. 500000");
}
}
```

```
package Guided2;

public class Main {
    public static void main(String[] args) {
        Hargatoken objectToken = new Hargatoken();
        objectToken.info();

        Hargapulsa objectPulsa = new Hargapulsa();
        objectPulsa.info();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Enkapsulasi ---
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.623 s
Finished at: 2026-10-12T22:10:19+07:00
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Jawaban :

```
package Unguided;

public class Dataset {

    public static final double BATAS_MISSING = 5.0;
    private static int totalDataset = 0;

    private String nama;
    private int jumlahBaris;
    private int jumlahKolom;
    private int jumlahMissing;

    public Dataset() {
        nama = "";
        jumlahBaris = 0;
        jumlahKolom = 0;
        jumlahMissing = 0;
        totalDataset++;
    }

    public Dataset(String nama) {
        this.nama = nama;
        jumlahBaris = 0;
        jumlahKolom = 0;
        jumlahMissing = 0;
        totalDataset++;
    }

  public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
jumlahMissing) {
        this.nama = nama;
        this.jumlahBaris = jumlahBaris;
        this.jumlahKolom = jumlahKolom;
        this.jumlahMissing = jumlahMissing;
        totalDataset++;
```

```
    }

Dataset.getTotalDataset()
    public static int getTotalDataset() {
        return totalDataset;
    }

    public String getNama() {
        return nama;
    }

    public int getJumlahBaris() {
        return jumlahBaris;
    }

    public int getJumlahKolom() {
        return jumlahKolom;
    }

    public int getJumlahMissing() {
        return jumlahMissing;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setJumlahBaris(int jumlahBaris) {
        if (jumlahBaris >= 0) {
            this.jumlahBaris = jumlahBaris;
        }
    }

    public void setJumlahKolom(int jumlahKolom) {
        if (jumlahKolom >= 0) {
            this.jumlahKolom = jumlahKolom;
        }
    }

    public void setJumlahMissing(int jumlahMissing) {
        if (jumlahMissing >= 0) {
            this.jumlahMissing = jumlahMissing;
        }
    }

    public double getPersentaseMissing() {
        int totalSel = jumlahBaris * jumlahKolom;

        if (totalSel == 0) {
            return 0;
        }
```

```
            return (double) jumlahMissing / totalSel * 100;
        }

        public boolean perluDibersihkan() {
            return getPersentaseMissing() > BATAS_MISSING;
        }
    }
```

```
package Unguided;
import java.util.Locale;
public class Laporandataset {

    public void cetak(Dataset d) {

        String status;
        if (d.perluDibersihkan()) {
            status = "Perlu dibersihkan";
        } else {
            status = "Bersih";
        }

        String persen = String.format(Locale.US, "%.2f", d.getPersentaseMissing());

        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama         : " + d.getNama());
        System.out.println("Jumlah Baris : " + d.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + d.getJumlahKolom());
        System.out.println("Missing      : " + d.getJumlahMissing() + " sel (" + persen +
"%)");
        System.out.println("Status       : " + status);
    }
}
```

```
package Unguided;

public class Main {

    public static void main(String[] args) {

        Dataset d1 = new Dataset();
        d1.setNama("Titanic");
        d1.setJumlahBaris(891);
        d1.setJumlahKolom(12);
        d1.setJumlahMissing(866);

        Dataset d2 = new Dataset("Wine Quality");

        Dataset d3 = new Dataset("Iris", 150, 5, 0);

        Dataset[] daftar = {d1, d2, d3};

        Laporandataset laporan = new Laporandataset();
```

```
        for (int i = 0; i < daftar.length; i++) {
            laporan.cetak(daftar[i]);
            System.out.println();
        }

        System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Enkapsulasi ---
=== Laporan Dataset ===
Nama          : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing       : 866 sel (8.10%)
Status        : Perlu dibersihkan

=== Laporan Dataset ===
Nama          : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing       : 0 sel (0.00%)
Status        : Bersih

=== Laporan Dataset ===
Nama          : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing       : 0 sel (0.00%)
Status        : Bersih

Total dataset dibuat : 3
------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------
Total time:  2.213 s
Finished at: 2026-10-12T22:20:46+07:00
------------------------------------------------------------
```

Penjelasan :

Program ini membuat tiga dataset lalu mencetak laporan kondisi masing-masing. Titanic punya 891 baris dan 12 kolom dengan 866 sel kosong (8,10%), jadi statusnya "Perlu dibersihkan". Wine Quality tercatat 0 baris dan 0 kolom sehingga dianggap "Bersih", padahal sebenarnya datanya kosong dan perlu dicek lagi. Iris punya 150 baris dan 5 kolom tanpa data hilang, jadi benar-benar bersih. Di akhir, program mencatat 3 dataset dibuat dan berhasil dijalankan (BUILD SUCCESS) dalam 2,2 detik.

D. Kesimpulan

Praktikum Modul 4 membahas enkapsulasi di Java, yaitu menyembunyikan atribut dengan `private` dan mengaksesnya lewat *getter* dan *setter*. Pada bagian guided, class `Manusia` memakai tiga *constructor* untuk membuat objek dengan data yang berbeda-beda, dan *package* dipakai untuk mengelompokkan class agar lebih rapi. Pada bagian unguided, class `Dataset` menerapkan enkapsulasi dengan validasi di *setter*, konstanta `BATAS_MISSING`, dan atribut `static` untuk menghitung total objek. Hasilnya, Titanic berstatus "Perlu dibersihkan" karena data kosongnya 8,10%, sedangkan Iris bersih. Dari praktikum ini dapat disimpulkan bahwa enkapsulasi membuat data lebih aman, kode lebih terstruktur, dan mudah digunakan kembali.